

iCar

Pipeline Microservices Complet

Rapport Technique — Projet AOS

Université UMBB — Faculté des Sciences

Formation Master 1 GL — Semestre 2

Matière AOS (Architecture Orienté Services)

Année 2025 / 2026

Membres du groupe

- 01 Berredjem Oussama
- 02 Ouriachi Abderrahmane Yacine
- 03 Sellami Siham
- 04 Boudabba Marwa

Contents

1	Contexte et Objectifs	4
1.1	Présentation du projet iCar	4
1.2	Domaine applicatif et justification	4
1.3	Besoins fonctionnels	4
1.4	Contraintes techniques	5
2	Architecture Microservices	5
2.1	Vue d'ensemble	5
2.2	Description des composants	6
2.2.1	Traefik — Reverse Proxy et Gateway	6
2.2.2	Consul — Service Registry	6
2.2.3	Auth Service	6
2.2.4	Catalog Service	6
2.2.5	Notification Service	7
2.2.6	RabbitMQ — Broker de messages	7
2.2.7	Endpoints de Production	7
2.3	Topologie des échanges RabbitMQ	7
2.4	Configuration RabbitMQ	8
2.5	Flux de communication détaillé	8
3	Implémentation — Auth Service	8
3.1	Modèle de données	8
3.2	Token JWT personnalisé	9
3.3	Configuration JWT	9
3.4	Endpoints de l'Auth Service	10
3.5	Gestion des permissions	10
4	Implémentation — Catalog Service	10
4.1	Modèle de données	10
4.1.1	UserModel — copie locale	10
4.1.2	VehiculeModel	10
4.1.3	VehiculeImage	11
4.2	Authentification stateless	11
4.3	Transaction atomique à la création	12
4.4	Endpoints du Catalog Service	13
4.5	Consumer RabbitMQ du Catalog Service	13
5	Implémentation — Notification Service	13
5.1	Modèle de données	13
5.1.1	Relations de suivi	14
5.2	Endpoints REST du Notification Service	14
5.3	Worker RabbitMQ	14
6	Sécurité	15
6.1	Contrôles d'accès	15
6.2	Validation des données	15
6.3	Sécurité de l'Infrastructure	15
6.3.1	Points d'attention identifiés	15
7	Déploiement Multi-Serveurs	16
7.1	Infrastructure — 6 instances	16

7.2	Accès aux Services d'Infrastructure	16
7.2.1	Détails des endpoints d'infrastructure	16
7.3	Stack technique	17
7.4	Infrastructure de Production — Captures d'écran	17
7.4.1	Tableau de bord Traefik	17
7.4.2	Service Registry Consul	19
7.4.3	RabbitMQ — Message Broker	20
7.5	Configuration Traefik — Routage et Middlewares	23
7.5.1	Entrypoints	23
7.5.2	Routers HTTP	23
7.5.3	Middlewares	23
7.6	Configuration Consul — Service Registry	24
7.6.1	Instances enregistrées dans Consul	25
7.7	RabbitMQ — Topologie des Exchanges et Queues	25
7.7.1	Exchanges configurés	25
7.7.2	Connexions actives	25
7.8	Architecture Complète — Récapitulatif	26
7.9	Procédure de déploiement	26
8	Tests et Validation	26
8.1	Collection Postman	26
8.2	Vérification de la communication asynchrone	27
9	Conclusion	27
9.1	Bilan technique	27
9.2	Grille d'évaluation	28
9.3	Points livrés	28

1. Contexte et Objectifs

1.1. Présentation du projet iCar

Présentation

iCar est une plateforme numérique de mise en relation entre acheteurs et vendeurs de véhicules d'occasion. Développée dans le cadre du module AOS, elle repose sur une architecture microservices complète, asynchrone et sécurisée, déployée sur plusieurs serveurs indépendants.

L'application offre à ses utilisateurs la possibilité de publier des annonces de véhicules enrichies de photos, de consulter le catalogue filtré, de suivre d'autres vendeurs ou des annonces spécifiques, et de recevoir des notifications en temps réel lors d'événements pertinents (nouvelle publication, mise à jour, suppression).

Le projet a été conçu pour répondre à l'ensemble des contraintes techniques imposées par le cahier des charges WAMS : isolation des services, communication asynchrone via broker de messages, registre de services, reverse proxy, et déploiement multi-serveurs.

1.2. Domaine applicatif et justification

Le marché de l'automobile d'occasion constitue un domaine applicatif riche en interactions utilisateurs et en besoins de scalabilité sélective. Trois raisons ont motivé ce choix :

1. **CRUD intensif** — la gestion de véhicules (création, modification, suppression, filtrage) génère un volume important de requêtes API qui justifie l'isolation du service catalogue.
2. **Gestion de médias** — l'upload de 1 à 15 images par annonce, avec image principale désignable et suppression physique automatique, illustre concrètement les problématiques de stockage fichier.
3. **Notifications asynchrones** — les alertes aux abonnés (followers d'un vendeur ou d'une annonce) sont un cas d'usage naturel pour la communication via broker de messages (RabbitMQ).

1.3. Besoins fonctionnels

Fonctionnalité	Description
Gestion des utilisateurs	Inscription, connexion sécurisée, rôles <i>user</i> et <i>admin</i> , profil modifiable, suppression de compte avec nettoyage en cascade
Catalogue véhicules	CRUD complet avec filtres, taxonomie en catégories et sous-catégories
Gestion d'images	Upload multi-fichiers (1–15), image principale, nettoyage physique auto
Système de follow	Suivre un utilisateur ou une annonce, se désabonner
Notifications	Alertes temps réel : nouveau post, mise à jour, nouveau follower
Synchronisation inter-services	Réplication des comptes utilisateurs via événements RabbitMQ

1.4. Contraintes techniques

Exigence WAMS	Solution retenue
Application REST métier	Catalog Service — endpoints CRUD + routes fonctionnelles
Service d'authentification	Auth Service — JWT HS256 avec rôles/permissions
Application UI/UX	Interface statique consommant l'API avec gestion des tokens
Communication asynchrone	RabbitMQ — échanges topic + queues directes durables
Service Registry	Consul pour la découverte dynamique des instances
Reverse Proxy	Traefik pour le routage frontal et l'équilibrage de charge
Déploiement multi-serveurs	Hugging Face Spaces — une instance par microservice

2. Architecture Microservices

2.1. Vue d'ensemble

Le projet iCar décompose l'application en trois microservices Django indépendants, chacun possédant deux instances déployées sur des serveurs distincts pour assurer haute disponibilité et tolérance aux pannes. L'architecture repose sur cinq composants principaux :

- **Traefik** — Reverse proxy et gateway frontal qui reçoit toutes les requêtes HTTP externes.
- **Consul** — Service registry pour la découverte dynamique des instances saines.
- **Services métier** — Auth, Catalog et Notification, chacun avec 2 instances et sa propre base de données.
- **RabbitMQ** — Broker de messages pour la communication asynchrone (découplé de Traefik).

Diagramme d'architecture — Production

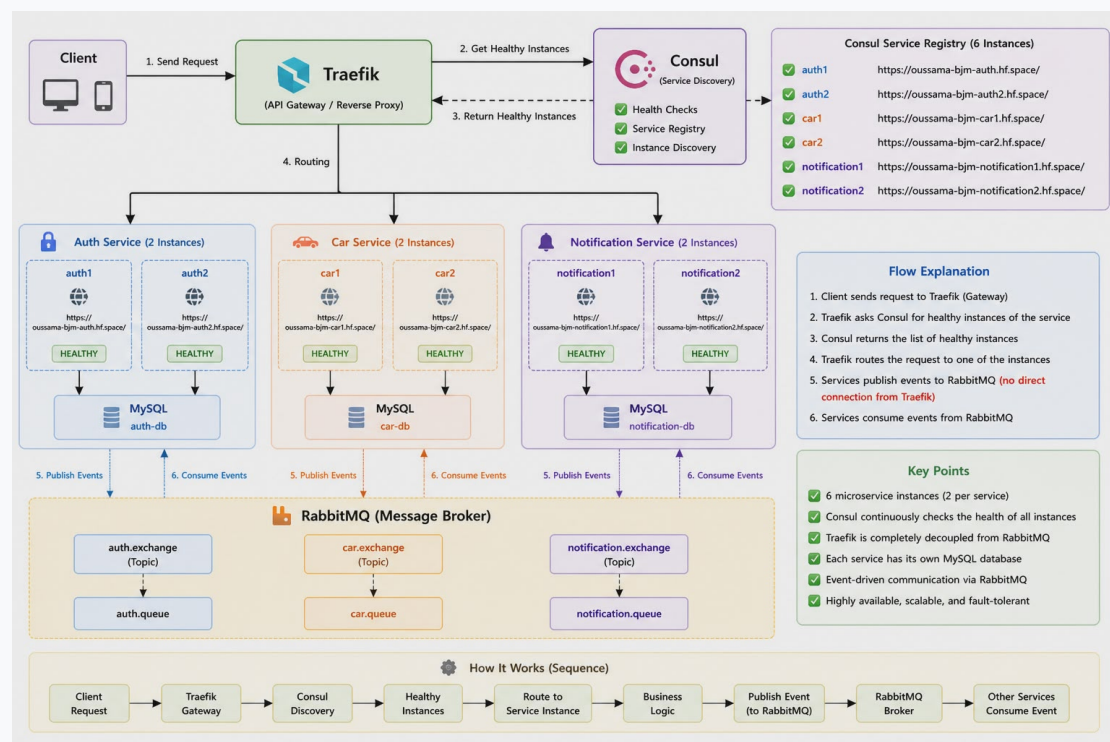


Figure 1: Architecture microservices iCar — Traefik, Consul, 6 instances et RabbitMQ

2.2. Description des composants

2.2.1. Traefik — Reverse Proxy et Gateway

Traefik agit comme point d'entrée unique (gateway) pour toutes les requêtes externes. Il interroge Consul pour obtenir la liste des instances saines et route dynamiquement le trafic vers les instances disponibles. Il est complètement découplé de RabbitMQ — son rôle se limite au routage HTTP synchrone.

2.2.2. Consul — Service Registry

Consul maintient un registre de tous les microservices et vérifie en continu la santé de chaque instance via des health checks. Lorsqu'une instance devient indisponible, elle est automatiquement retirée du pool. Traefik interroge Consul pour découvrir dynamiquement les endpoints sains.

2.2.3. Auth Service

Service Django dédié à la gestion des identités. Déployé en 2 instances sur Hugging Face Spaces :

- <https://oussama-bjm-auth.hf.space/>
- <https://oussama-bjm-auth2.hf.space/>

Il est le seul service à accéder à la table d'utilisateurs réelle et à émettre des tokens JWT. Il publie sur l'échange `user_events` (topic) lors de toute mutation d'un compte utilisateur. Chaque instance possède sa propre base de données SQLite.

2.2.4. Catalog Service

Cœur fonctionnel de la marketplace. Déployé en 2 instances :

- <https://oussama-bjm-car1.hf.space/>

- <https://oussama-bjm-car2.hf.space/>

Il expose l'API REST métier pour les véhicules, les catégories et les images. Il maintient une copie locale de la table des utilisateurs (synchronisée via RabbitMQ) afin de gérer les clés étrangères sans appel inter-service au moment de la requête.

2.2.5. Notification Service

Service travailleur qui écoute plusieurs queues et exchanges RabbitMQ pour créer des notifications en base de données. Déployé en 2 instances :

- <https://oussama-bjm-notification1.hf.space/>
- <https://oussama-bjm-notification2.hf.space/>

Il gère également les relations *follow user* et *follow post* via ses propres endpoints REST.

2.2.6. RabbitMQ — Broker de messages

Le broker central assure le découplage temporel et spatial entre producteurs et consommateurs. Il est complètement découplé de Traefik — les services publient et consomment des événements directement, sans passer par la gateway.

2.2.7. Endpoints de Production

Composant	Endpoint
Gateway (Traefik)	http://api.icaralgerie.com:8080
Service Registry (Consul)	http://api.icaralgerie.com:8500
Message Broker (RabbitMQ)	amqp://guest:guest@147.224.161.221:5672

Haute Disponibilité

Avec 2 instances par service, l'architecture tolère la panne d'une instance sans interruption de service. Consul détecte automatiquement l'instance défaillante et Traefik cesse de lui envoyer du trafic. RabbitMQ assure la persistance des messages, évitant toute perte lors d'un redémarrage.

2.3. Topologie des échanges RabbitMQ

Exchange / Queue	Type	Routing key	Consommateur
<code>user_events</code>	topic	<code>user.created</code> , <code>user.updated</code> , <code>user.deleted</code>	Catalog + Notif
<code>vehicle_events</code>	topic	<code>vehicle.deleted</code>	Notification
<code>content_event</code>	direct	<code>new_post</code> , <code>update_car</code>	Notification
<code>follow_user</code>	direct	<code>follower_id</code> , <code>following_id</code>	Notification
<code>follow_post</code>	direct	<code>user_id</code> , <code>post_id</code>	Notification

2.4. Configuration RabbitMQ

Paramètre	Valeur
Hôte	147.224.161.221
Port AMQP	5672
Port Management	15672 (optionnel)
Nom d'utilisateur	guest
Mot de passe	guest
Sécurité	Non TLS (port 443 non configuré)

Note de sécurité RabbitMQ

L'utilisation des identifiants `guest/guest` et l'absence de TLS constituent une limitation de sécurité. Pour un passage en production réelle, il est impératif de :

1. Changer le mot de passe par défaut
2. Configurer TLS sur le port 5671 (AMQPS)
3. Restreindre l'accès réseau via firewall
4. Utiliser des utilisateurs dédiés par service (principe de moindre privilège)

2.5. Flux de communication détaillé

La séquence complète d'une requête est illustrée ci-dessous :

1. **Client** envoie une requête HTTP à **Traefik** (gateway).
2. **Traefik** interroge **Consul** pour obtenir la liste des instances saines du service cible.
3. **Consul** retourne la liste des instances saines.
4. **Traefik** route la requête vers une instance spécifique (load balancing).
5. Le service traite la logique métier et, si nécessaire, **publie un événement** vers **RabbitMQ**.
6. **RabbitMQ** distribue l'événement à tous les services **consommateurs** abonnés (asynchrone, sans intervention de Traefik).

Remarque importante : Traefik n'intervient que pour le trafic HTTP synchrone (requêtes API). Les événements RabbitMQ circulent directement entre services, garantissant un découplage total et une meilleure performance.

3. Implémentation — Auth Service

3.1. Modèle de données

L'Auth Service définit un modèle utilisateur personnalisé héritant de `AbstractBaseUser` et `PermissionsMixin`, avec `email` comme identifiant principal.

Listing 1: Modèle User — `icar_auth/users/models.py`

```
1 class User(AbstractBaseUser, PermissionsMixin):
2     ROLE_CHOICES = [('user', 'User'), ('admin', 'Admin')]
3     id = models.AutoField(primary_key=True)
```

```

4     name      = models.CharField(max_length=255)
5     email     = models.EmailField(unique=True)
6     number    = models.CharField(max_length=20, blank=True, null=True)
7
8     role      = models.CharField(max_length=10,
9                                 choices=ROLE_CHOICES, default='user')
10    is_active  = models.BooleanField(default=True)
11    is_staff   = models.BooleanField(default=False)
12    date_joined = models.DateTimeField(default=timezone.now)
13    last_login = models.DateTimeField(null=True, blank=True)
14    USERNAME_FIELD = 'email'
15    REQUIRED_FIELDS = ['name']

```

3.2. Token JWT personnalisé

Un token JWT standard ne contient que le `user_id`. Afin d'éviter un appel à l'Auth Service dans chaque requête des autres services, nous avons étendu `RefreshToken` pour injecter le champ `name` dans le payload.

Listing 2: Token personnalisé — users/tokens.py

```

1 class CustomRefreshToken(RefreshToken):
2     @classmethod
3     def for_user(cls, user):
4         token = super().for_user(user)
5         token['name'] = user.name
6         token.access_token['name'] = user.name
7         return token

```

3.3. Configuration JWT

Paramètre	Valeur / Description
<code>ACCESS_TOKEN_LIFETIME</code>	1 jour
<code>REFRESH_TOKEN_LIFETIME</code>	2 jours
<code>ROTATE_REFRESH_TOKEN</code>	<code>True</code> — nouveau refresh à chaque renouvellement
<code>BLACKLIST_AFTER_ROTATION</code>	<code>True</code> — l'ancien refresh est blacklisté
<code>ALGORITHM</code>	HS256
<code>USER_ID_CLAIM</code>	<code>user_id</code>
<code>AUTH_HEADER_TYPES</code>	<code>Bearer</code>

3.4. Endpoints de l'Auth Service

Méthode	URL	Description
POST	/signup/	Création de compte + event RabbitMQ
POST	/login/	Auth email/password → access + refresh tokens
POST	/logout/	Blackliste le refresh token
POST	/token/refresh/	Renouvellement du token
GET/PUT	/profile/	Consultation et modification du profil
POST	/change-password/	Changement de mot de passe
POST	/forgot-password/	Envoi d'un lien de réinitialisation
POST	/reset-password/	Réinitialisation du mot de passe
GET	/users/	Liste des utilisateurs (admin uniquement)
GET/PUT/DELETE	/users/<id>/	Gestion d'un utilisateur
POST	/admin/reset-password/<id>/	Reset admin du mot de passe
DELETE	/users/me/delete/	Suppression du compte + cascade RabbitMQ

3.5. Gestion des permissions

Deux classes de permission sont définies :

- `IsAdmin` — vérifie `request.user.role == 'admin'` au niveau requête et objet.
- `IsUserOrAdmin` — autorise si l'utilisateur est admin ou s'il s'agit de son propre objet (`obj.id == request.user.id`).

4. Implémentation — Catalog Service

4.1. Modèle de données

4.1.1. UserModel — copie locale

Listing 3: UserModel local — réplique depuis Auth Service

```

1 class UserModel(models.Model):
2     id = models.IntegerField(primary_key=True)
3     name = models.CharField(max_length=255)
4     email = models.EmailField()
5     number = models.CharField(max_length=20, blank=True, null=True)
6     role = models.CharField(max_length=10, default='user')
7     date_joined = models.DateTimeField()
8     class Meta:
9         db_table = 'users'

```

4.1.2. VehiculeModel

Listing 4: VehiculeModel — champs métier complets

```

1 class VehiculeModel(models.Model):
2     name = models.TextField()
3     brand = models.TextField()
4     subcategory = models.ForeignKey(SubCategorieModel,
5                                     on_delete=models.CASCADE)
6     user = models.ForeignKey(UserModel,
7                              on_delete=models.CASCADE,
8                              related_name='vehicles')
9     year = models.PositiveIntegerField(
10         validators=[MinValueValidator(1800),
11                    MaxValueValidator(2027)])
12     status = models.CharField(choices=Status.choices,
13                               default=Status.ACTIVE, max_length=20)
14     description = models.TextField(max_length=400)
15     phone = models.CharField(max_length=10)
16     location = models.CharField(max_length=255)
17     milage = models.PositiveBigIntegerField()
18     couleur = models.CharField(max_length=100)
19     fuel = models.CharField(choices=FuelType.choices,
20                             max_length=20)
21     transmissionType = models.CharField(choices=TransmissionType.
22                                         choices,
23                                         max_length=30)
24     created_at = models.DateTimeField(auto_now_add=True)
25     updated_at = models.DateTimeField(auto_now=True)

```

4.1.3. VehiculeImage

Listing 5: VehiculeImage — suppression physique du fichier

```

1 class VehiculeImage(models.Model):
2     vehicle = models.ForeignKey(VehiculeModel,
3                                 on_delete=models.CASCADE, related_name='images')
4     image = models.ImageField(
5         upload_to='vehicle_images/%Y/%m/%d/')
6     uploaded_at = models.DateTimeField(auto_now_add=True)
7     is_primary = models.BooleanField(default=False)
8
9     def delete(self, *args, **kwargs):
10         if self.image and os.path.isfile(self.image.path):
11             os.remove(self.image.path)
12         super().delete(*args, **kwargs)
13
14 @receiver(pre_delete, sender=VehiculeImage)
15 def delete_vehicle_image_file(sender, instance, **kwargs):
16     if instance.image and os.path.isfile(instance.image.path):
17         os.remove(instance.image.path)

```

4.2. Authentication stateless

Listing 6: StatelessJWTAuthentication

```

1 class StatelessJWTAuthentication(authentication.BaseAuthentication):
2     def authenticate(self, request):
3         auth_header = request.headers.get('Authorization', '')
4         token = auth_header.replace('Bearer ', '')
5         if not token:
6             return None

```

```
7         try:
8             access_token = AccessToken(token)
9             user_id = access_token.payload.get('user_id')
10            user_name = access_token.payload.get('name')
11            user = type('User', (), {
12                'id': user_id, 'name': user_name,
13                'is_authenticated': True
14            })()
15            return (user, token)
16        except (InvalidToken, TokenError) as e:
17            raise exceptions.AuthenticationFailed(str(e))
```

4.3. Transaction atomique à la création

Listing 7: Création de véhicule avec transaction atomique

```
1 @transaction.atomic
2 def create(self, validated_data):
3     images_data = validated_data.pop('images', [])
4     primary_index = validated_data.pop('primary_image_index', None)
5     vehicle = VehiculeModel.objects.create(**validated_data)
6     try:
7         for index, image_data in enumerate(images_data):
8             is_primary = (primary_index == index) if \
9                 primary_index is not None else (index == 0)
10            VehiculeImage.objects.create(vehicle=vehicle,
11                                         image=image_data,
12                                         is_primary=is_primary)
13        except Exception as e:
14            for img in vehicle.images.all():
15                img.delete()
16            raise serializers.ValidationError(
17                f"Error uploading images: {str(e)}")
18    return vehicle
```

4.4. Endpoints du Catalog Service

Méthode	URL	Description
GET	/api/vehicules/	Liste publique — filtrée sur ACTIVE
POST	/api/vehicules/	Création — multipart/form-data, auth requise
GET	/api/vehicules/{id}/	Détail d'un véhicule
PUT/PATCH	/api/vehicules/{id}/	Mise à jour — propriétaire uniquement
DELETE	/api/vehicules/{id}/	Suppression + event <code>vehicle.deleted</code>
GET	/api/vehicules/my_cars/	Annonces de l'utilisateur connecté
POST	/api/vehicules/{id}/add_image/	Ajouter une image
DELETE	/api/vehicules/{id}/remove-image/{img_id}/	Supprimer une image
PATCH	/api/vehicules/{id}/set_primary_image/	Changer l'image principale
GET/POST/PUT/DELETE	/api/categories/	Gestion des catégories
GET/POST/PUT/DELETE	/api/subcategories/	Gestion des sous-catégories

4.5. Consumer RabbitMQ du Catalog Service

Le consumer (lancé via `python manage.py consume_user_events`) écoute l'échange `user_events` sur les trois routing keys :

- `user.created` → `get_or_create()` avec tous les champs.
- `user.updated` → mise à jour sélective des champs modifiés.
- `user.deleted` → suppression + cascade sur tous les véhicules et images.

5. Implémentation — Notification Service

5.1. Modèle de données

Listing 8: Modèle Notification

```

1 class Notification(models.Model):
2     class Type(models.TextChoices):
3         FOLLOW_USER = "follow_user"
```

```

4 FOLLOW_POST = "follow_post"
5 POST       = "post"
6
7 user       = models.ForeignKey(UserModel,
8                               related_name="notifications",
9                               on_delete=models.CASCADE)
10 subject    = models.CharField(max_length=255)
11 message    = models.TextField()
12 is_read    = models.BooleanField(default=False)
13 created_at = models.DateTimeField(auto_now_add=True)
14 post       = models.ForeignKey(VehiculeModel, null=True,
15                               blank=True, on_delete=models.CASCADE)
16 follower   = models.ForeignKey(UserModel, null=True,
17                               blank=True, related_name="sent_notifications",
18                               on_delete=models.CASCADE)
19 type       = models.CharField(max_length=20, choices=Type.choices)
20 class Meta:
21     ordering = ['-created_at']

```

5.1.1. Relations de suivi

- `UserFollowPost` — contrainte d'unicité (`user`, `post`).
- `UserFollowUser` — contrainte d'unicité (`follower`, `following`) + `CheckConstraint` pour interdire l'auto-follow.

5.2. Endpoints REST du Notification Service

Méthode	URL	Description
GET	<code>/notifications/</code>	Notifications de l'utilisateur
PATCH	<code>/notifications/<pk>/read/</code>	Marquer comme lu
PATCH	<code>/notifications/read-all/</code>	Marquer toutes comme lues
DELETE	<code>/notifications/<pk>/</code>	Supprimer une notification
POST	<code>/follow/post/</code>	Suivre une annonce
DELETE	<code>/unfollow/post/</code>	Se désabonner d'une annonce
GET	<code>/follow/post/list/</code>	Lister ses abonnements annonces
POST	<code>/follow/user/</code>	Suivre un utilisateur
DELETE	<code>/unfollow/user/</code>	Se désabonner d'un utilisateur
GET	<code>/follow/user/list/</code>	Lister ses abonnements utilisateurs

5.3. Worker RabbitMQ

Le worker est lancé via `python manage.py start_worker` et écoute simultanément cinq sources de messages :

1. `follow_user` — crée une notification pour l'utilisateur suivi.
2. `follow_post` — confirmation à l'abonné + notification au propriétaire de l'annonce.
3. `content_event` — fan-out vers tous les followers via `bulk_create`.
4. Queue `notification_user_events` — synchronisation des utilisateurs locaux.

5. Queue `notification_vehicle_events` — nettoyage en cascade lors de la suppression d'un véhicule.

Optimisation

Le fan-out utilise `Notification.objects.bulk_create()` plutôt qu'une boucle `save()` individuelle, ce qui réduit le nombre de requêtes SQL à une seule insertion groupée, quelle que soit la taille de l'audience.

6. Sécurité

6.1. Contrôles d'accès

Règle	Implémentation
Propriété véhicule	<code>str(instance.user_id) != str(request.user.id) → HTTP 403</code>
Admin only	Classe <code>IsAdmin</code> — vérifie <code>role == 'admin'</code>
Propriétaire ou admin	Classe <code>IsUserOrAdmin</code> avec <code>has_object_permission()</code>
Anti-self-follow	<code>CheckConstraint</code> DB + vérification applicative
Logout blacklist	Le refresh token est blacklisté à la déconnexion

6.2. Validation des données

Champ	Règle de validation
email	Unique, validé par Django <code>EmailField</code> + normalisation
password	Django <code>validate_password</code> (longueur, complexité)
year	Entre 1800 et 2027 (<code>MinValueValidator</code> / <code>MaxValueValidator</code>)
images	1 à 15 fichiers, taille max 1 Mo par fichier
fuel	gasoil, diesel, hybrid, electric
transmissionType	MANUAL, AUTOMATIC, SEMI_AUTOMATIC, CVT
status	DRAFT, ACTIVE, INACTIVE

6.3. Sécurité de l'Infrastructure

6.3.1. Points d'attention identifiés

Point faible	Impact	Recommandation
RabbitMQ guest/guest	Accès non authentifié au broker	Changer credentials + utilisateurs dédiés
Absence de TLS sur RabbitMQ	Ècoute possible des messages	Configurer AMQPS (port 5671)
Ports exposés (8500, 8080, 5672)	Surface d'attaque augmentée	Restreindre par firewall / VPN
Consul sans ACL	Découverte non authentifiée	Activer ACL + TLS

7. Déploiement Multi-Serveurs

7.1. Infrastructure — 6 instances

L'architecture de production se compose de **6 instances** réparties sur trois microservices, chacun avec sa propre base de données SQLite (voir sous-section 7.4).

Service	Instances (URLs de production)
Auth Service	https://oussama-bjm-auth.hf.space/ (auth1)
	https://oussama-bjm-auth2.hf.space/ (auth2)
Catalog Service	https://oussama-bjm-car1.hf.space/ (car1)
	https://oussama-bjm-car2.hf.space/ (car2)
Notification Service	https://oussama-bjm-notification1.hf.space/ (notification1)
	https://oussama-bjm-notification2.hf.space/ (notification2)

7.2. Accès aux Services d'Infrastructure

Les services d'infrastructure sont accessibles aux adresses suivantes :

Service	URL / Adresse	Port
Consul (Service Registry)	api.icaralgerie.com	8500
Traefik (Dashboard / API)	api.icaralgerie.com	8080
RabbitMQ (Broker)	147.224.161.221	5672 (AMQP)

7.2.1. Détails des endpoints d'infrastructure

- **Consul API** : <http://api.icaralgerie.com:8500/v1/catalog/services>
- **Consul Health Checks** : <http://api.icaralgerie.com:8500/v1/health/service/<service-name>>
- **Traefik Dashboard** : <http://api.icaralgerie.com:8080/dashboard/>
- **RabbitMQ Management (UI)** : <http://147.224.161.221:15672> (si activé)

7.3. Stack technique

Composant	Technologie
Framework	Django 6.0.3 + Django REST Framework
Gateway / Reverse Proxy	Traefik
Service Registry	Consul (avec health checks)
Auth JWT	djangorestframework-simplejwt
Messagerie asynchrone	RabbitMQ + Pika (client Python)
Images	Pillow
CORS	django-cors-headers
Base de données	SQLite (instance par service)

7.4. Infrastructure de Production — Captures d'écran

Les captures d'écran ci-dessous illustrent l'état réel de l'infrastructure déployée.

7.4.1. Tableau de bord Traefik

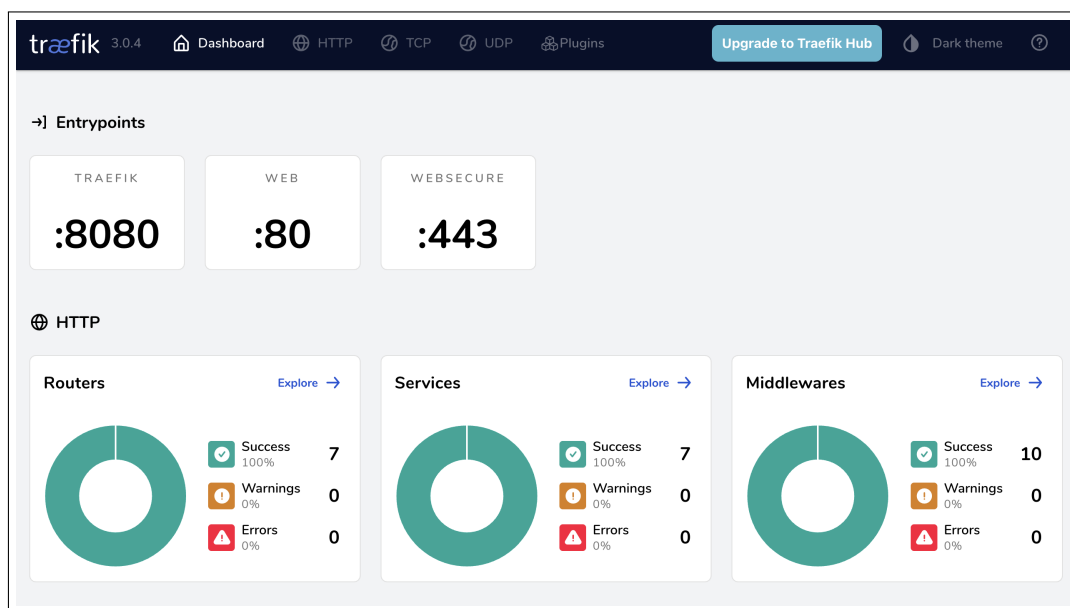
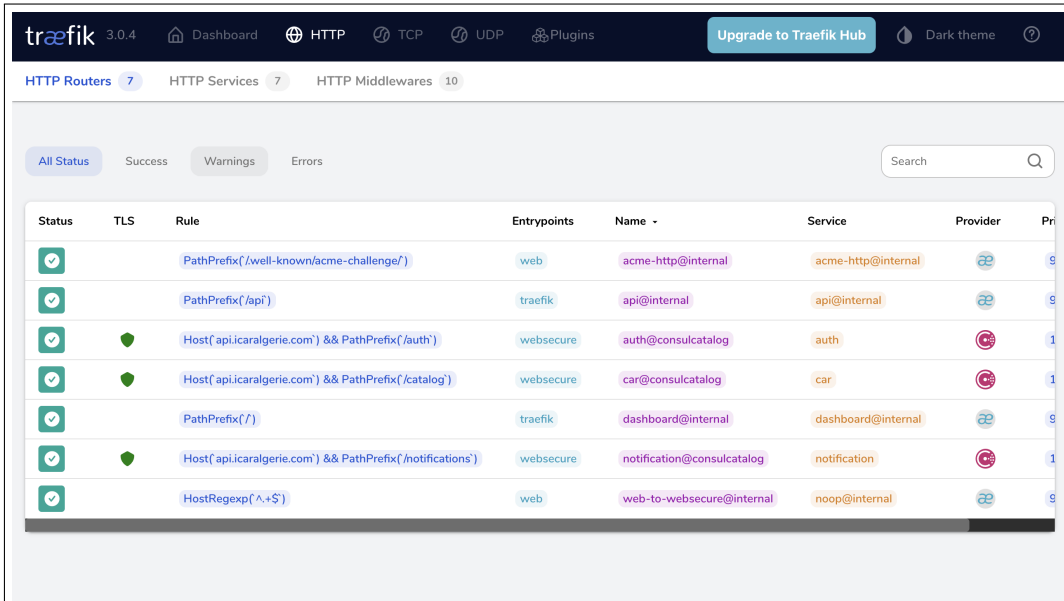


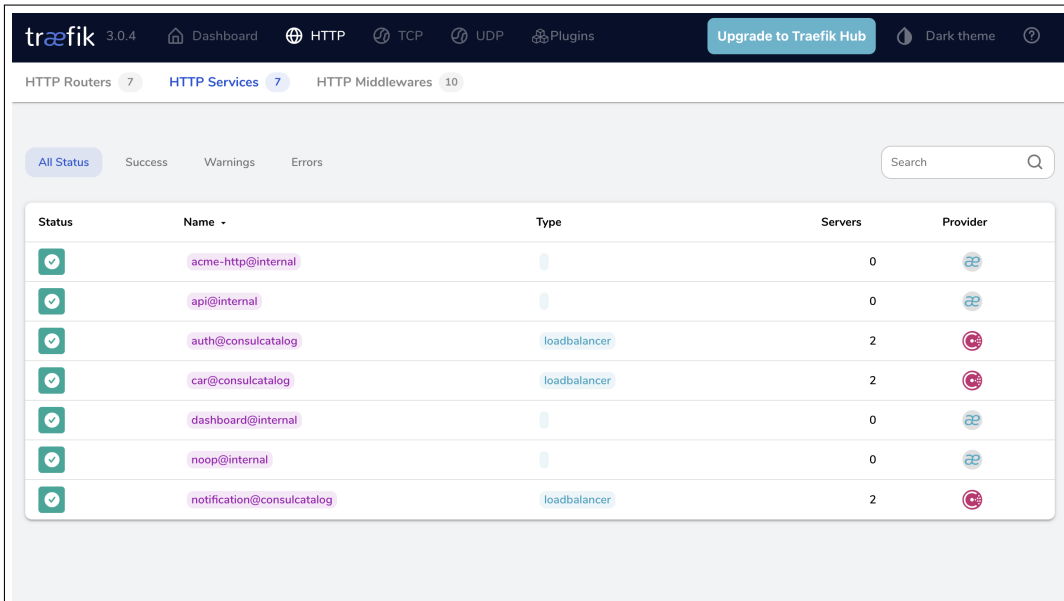
Figure 2: Tableau de bord Traefik — Entrypoints et statistiques



The screenshot shows the Traefik dashboard with the 'HTTP Routers' tab selected. It displays a list of 7 active routers. Each row includes a status icon (green checkmark), a TLS status (green shield), a rule, an endpoint, a name, a service, a provider, and a priority.

Status	TLS	Rule	Endpoints	Name	Service	Provider	Priority
✓		PathPrefix(`/well-known/acme-challenge/`)	web	acme-http@internal	acme-http@internal	acme	9
✓		PathPrefix(`/api`)	traefik	api@internal	api@internal	acme	9
✓	✓	Host(`api.icaralgerie.com`) && PathPrefix(`/auth`)	websecure	auth@consulcatalog	auth	consulcatalog	1
✓	✓	Host(`api.icaralgerie.com`) && PathPrefix(`/catalog`)	websecure	car@consulcatalog	car	consulcatalog	1
✓		PathPrefix(`/`)	traefik	dashboard@internal	dashboard@internal	acme	9
✓	✓	Host(`api.icaralgerie.com`) && PathPrefix(`/notifications`)	websecure	notification@consulcatalog	notification	consulcatalog	1
✓		HostRegexp(`^.+`)	web	web-to-websecure@internal	noop@internal	acme	9

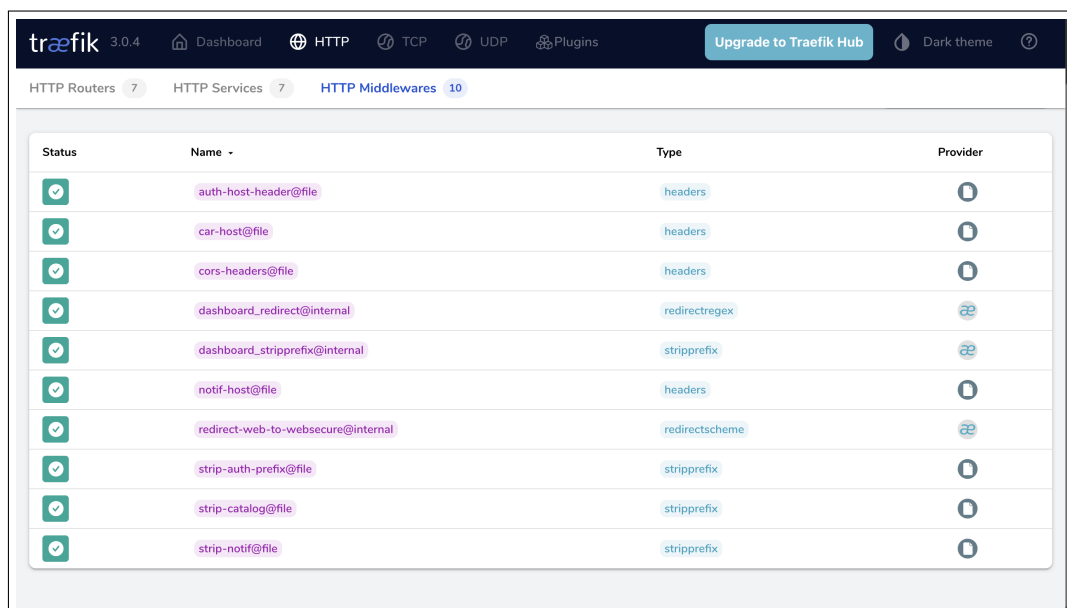
Figure 3: Routers HTTP configurés dans Traefik (7 routers actifs)



The screenshot shows the Traefik dashboard with the 'HTTP Services' tab selected. It displays a list of 7 services. Each row includes a status icon (green checkmark), a name, a type, the number of servers, and a provider.

Status	Name	Type	Servers	Provider
✓	acme-http@internal		0	acme
✓	api@internal		0	acme
✓	auth@consulcatalog	loadbalancer	2	consulcatalog
✓	car@consulcatalog	loadbalancer	2	consulcatalog
✓	dashboard@internal		0	acme
✓	noop@internal		0	acme
✓	notification@consulcatalog	loadbalancer	2	consulcatalog

Figure 4: Services load balancer — 2 instances par service métier

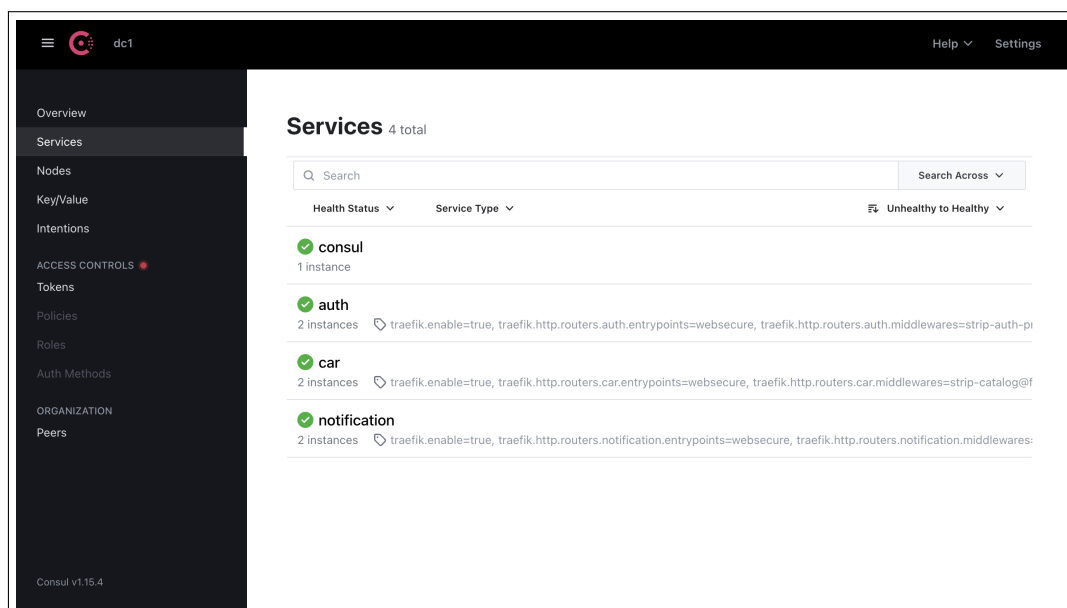


The screenshot shows the Traefik dashboard interface. At the top, there's a navigation bar with 'traefik 3.0.4', a 'Dashboard' link, and tabs for 'HTTP', 'TCP', 'UDP', and 'Plugins'. A button 'Upgrade to Traefik Hub' is on the right. Below the navigation bar, there are tabs for 'HTTP Routers' (7), 'HTTP Services' (7), and 'HTTP Middlewares' (10). The 'HTTP Middlewares' tab is selected, displaying a table of 10 configured middlewares.

Status	Name	Type	Provider
✓	auth-host-header@file	headers	📄
✓	car-host@file	headers	📄
✓	cors-headers@file	headers	📄
✓	dashboard_redirect@internal	redirectregex	⚙️
✓	dashboard_stripprefix@internal	stripprefix	⚙️
✓	notif-host@file	headers	📄
✓	redirect-web-to-websecure@internal	redirectscheme	⚙️
✓	strip-auth-prefix@file	stripprefix	📄
✓	strip-catalog@file	stripprefix	📄
✓	strip-notif@file	stripprefix	📄

Figure 5: Middlewares Traefik (10 middlewares configurés)

7.4.2. Service Registry Consul



The screenshot shows the Consul dashboard interface. On the left is a sidebar with navigation links: Overview, Services, Nodes, Key/Value, Intentions, ACCESS CONTROLS, Tokens, Policies, Roles, Auth Methods, ORGANIZATION, and Peers. The 'Services' link is selected. The main area shows 'Services 4 total'. There's a search bar and filters for 'Health Status' and 'Service Type'. A list of services is displayed, each with a green checkmark icon, a name, and a count of instances. The services are: consul (1 instance), auth (2 instances), car (2 instances), and notification (2 instances). Each service has a link to its configuration page.

Service	Instances	Configuration
consul	1 instance	
auth	2 instances	traefik.enable=true, traefik.http.routers.auth.entrypoints=websecure, traefik.http.routers.auth.middlewares=strip-auth-pi
car	2 instances	traefik.enable=true, traefik.http.routers.car.entrypoints=websecure, traefik.http.routers.car.middlewares=strip-catalog@f
notification	2 instances	traefik.enable=true, traefik.http.routers.notification.entrypoints=websecure, traefik.http.routers.notification.middlewares:

Figure 6: Consul — Vue d'ensemble des services et instances enregistrées

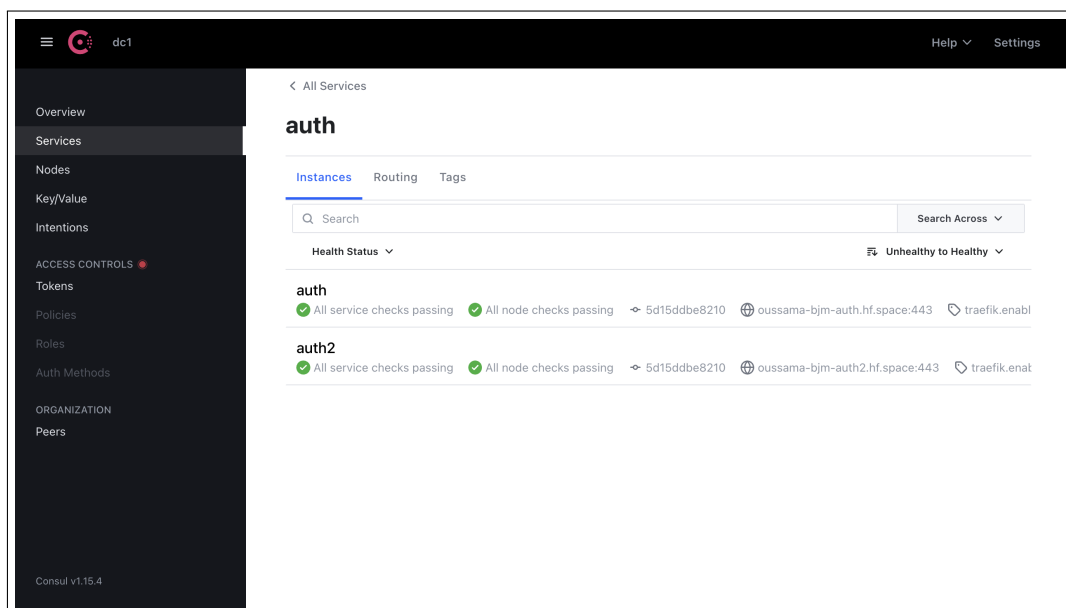


Figure 7: Consul — Détail du service Auth avec ses 2 instances saines

7.4.3. RabbitMQ — Message Broker

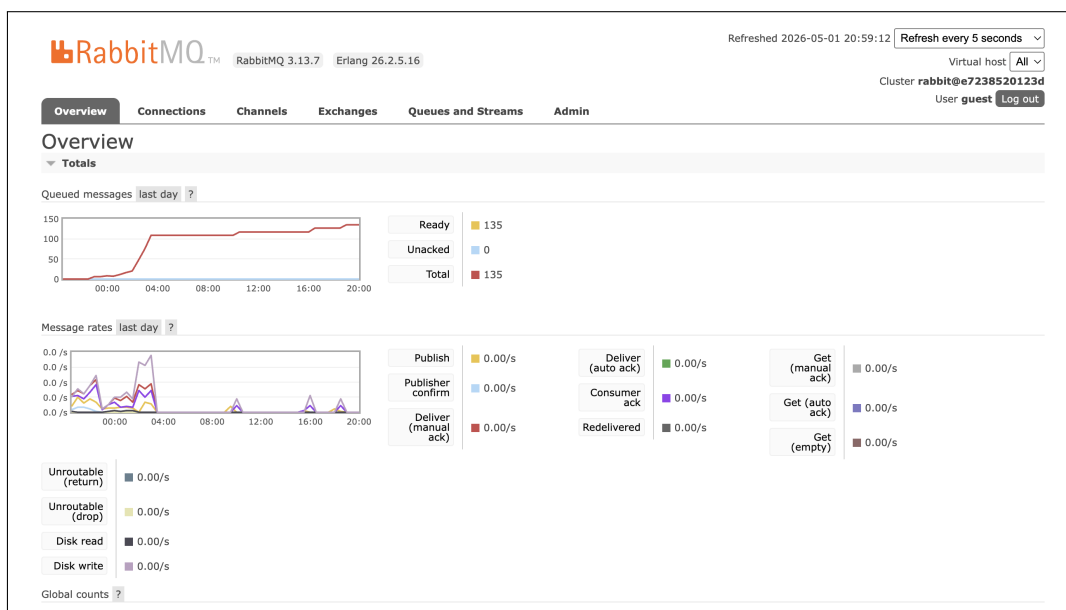


Figure 8: RabbitMQ — Vue d'ensemble avec métriques globales

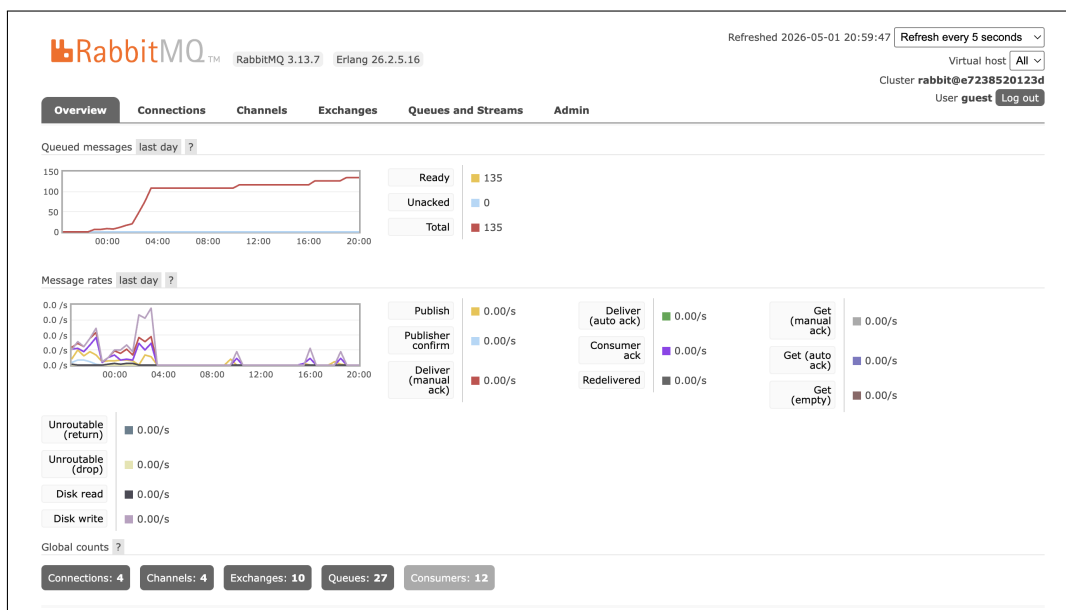


Figure 9: RabbitMQ — Files d'attente (Queues) et taux de messages

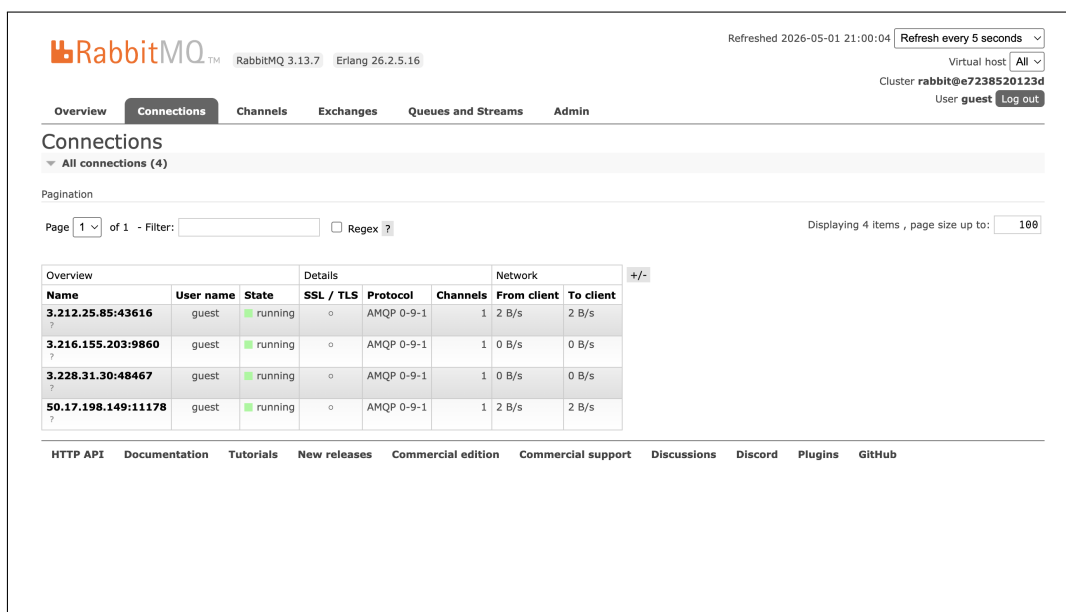


Figure 10: RabbitMQ — Connexions actives (4 connections de services)

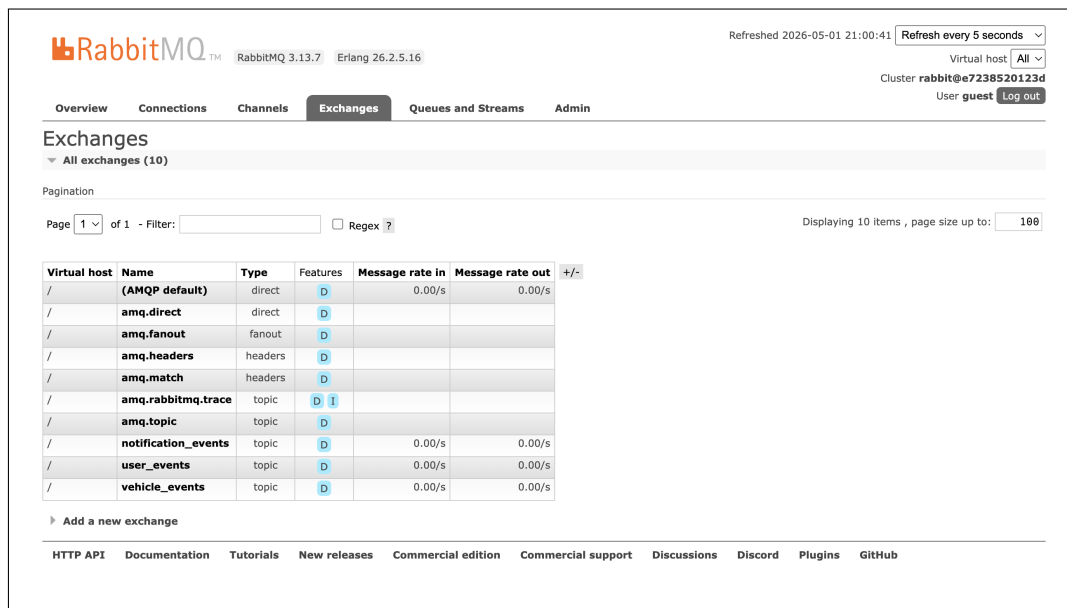


Figure 11: RabbitMQ — Exchanges (user_events, vehicle_events, notification_events)

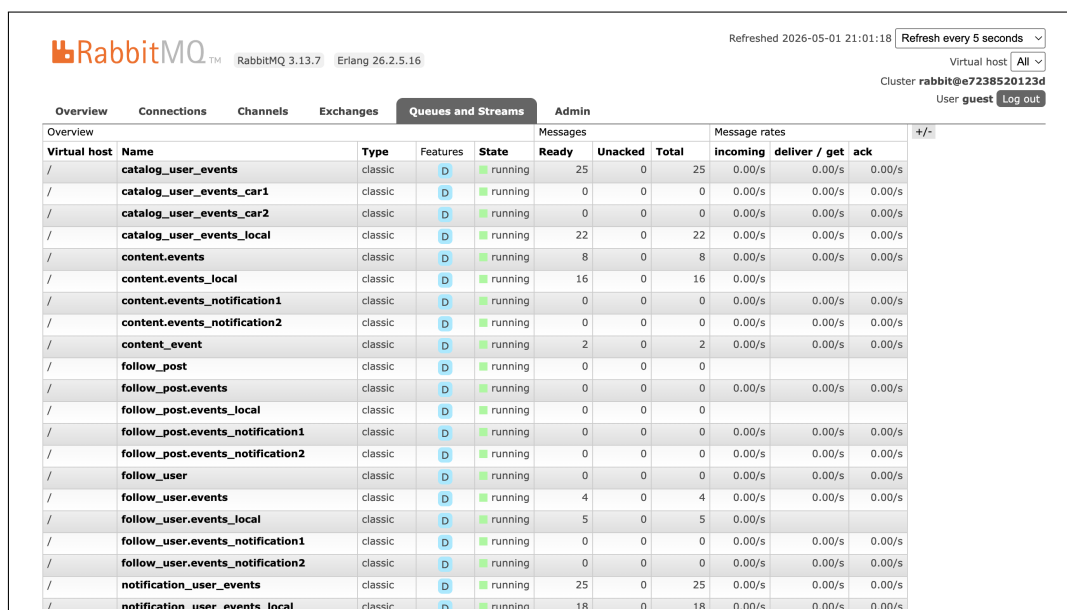


Figure 12: Détail des files d'attente RabbitMQ — messages par queue

Analyse de l'état RabbitMQ

- **Version:** RabbitMQ 3.13.7 avec Erlang 26.2.5.16
- **Connections actives:** 4 connexions (services Auth, Catalog, Notification)
- **Channels:** 4 channels ouverts
- **Queues durables:** 27 queues configurées avec persistance (feature 'D')
- **Consumers:** 12 consommateurs actifs répartis sur les 3 services
- **Exchanges personnalisés:**
 - user_events (topic) - synchronisation utilisateurs
 - vehicle_events (topic) - événements véhicules

- notification_events (topic) - notifications
- **Queues principales:**
 - catalog_user_events : 25 messages (sync utilisateurs)
 - catalog_user_events_local : 22 messages
 - content.events_local : 16 messages
 - content.events : 8 messages
- **Isolation par instance:** Chaque instance possède sa queue dédiée (..._car1, ..._car2, ..._notification1, ..._notification2)
- **Messages en attente total:** 135 Ready, 0 Unacked (traitement intégral)
- **Taux de publication:** stable avec des pics pendant les phases de test

7.5. Configuration Traefik — Routage et Middlewares

Traefik version 3.0.4 est déployé comme gateway frontale avec la configuration suivante :

7.5.1. Entrypoints

Entrypoint	Port	Utilisation
TRAEFIK	8080	Dashboard et API de gestion
WEB	80	Trafic HTTP non sécurisé
WEBSECURE	443	Trafic HTTPS sécurisé

7.5.2. Routers HTTP

L'infrastructure expose **7 routers HTTP** configurés comme suit :

Router	Rule	Entrypoint	Service
acme-http	PathPrefix(/well-known/acme-challenge/)	web	acme-http@internal
api	PathPrefix(/api)	traefik	api@internal
auth	Host(api.icaralgerie)&& PathPrefix(/auth)	websecure	auth@consulcatalog
car	Host(api.icaralgerie)&& PathPrefix(/catalog)	websecure	car@consulcatalog
dashboard	PathPrefix(/)	traefik	dashboard@internal
notification	Host(api.icaralgerie)&& PathPrefix(/notifications)	websecure	notification@consulcatalog
redirect-web	HostRegexp({.+})	web	web-to-websecure@internal

7.5.3. Middlewares

10 middlewares sont configurés pour la manipulation des requêtes :

Middleware	Type	Fonction
auth-host-header@file	headers	Ajout des en-têtes d'authentification
car-host@file	headers	Configuration des en-têtes pour Catalog
cors-headers@file	headers	Gestion CORS
dashboard_redirect@i	redirectregex	Redirection du dashboard
dashboard_stripprefi	stripprefix	Nettoyage du préfixe dashboard
notif-host@file	headers	En-têtes pour Notification Service
redirect-web-to-websecure@internal	redirectscheme	HTTP → HTTPS
strip-auth-prefix@file	stripprefix	Suppression du préfixe /auth
strip-catalog@file	stripprefix	Suppression du préfixe /catalog
strip-notif@file	stripprefix	Suppression du préfixe /notifications

Note sur la configuration Traefik

La découverte des services est réalisée via **Consul Catalog** (@consulcatalog), permettant l'ajout/suppression dynamique des instances sans redémarrage.

7.6. Configuration Consul — Service Registry

Consul version **v1.15.4** est déployé dans le datacenter **dc1** avec les services suivants :

- **consul** : 1 instance (service registry lui-même)
- **auth** : 2 instances enregistrées
 - Tags : traefik.enable=true
 - Entrypoint : websecure
 - Middleware : strip-auth-prefix@file
- **car** : 2 instances enregistrées
 - Tags : traefik.enable=true
 - Entrypoint : websecure
 - Middleware : strip-catalog@file
- **notification** : 2 instances enregistrées
 - Tags : traefik.enable=true
 - Entrypoint : websecure
 - Middleware : strip-notif@file

Toutes les instances ont un statut "**All service checks passing**" et "**All node checks passing**".

7.6.1. Instances enregistrées dans Consul

Service	Instance	URL
auth	auth1	https://oussama-bjm-auth.hf.space:443
auth	auth2	https://oussama-bjm-auth2.hf.space:443
car	car1	https://oussama-bjm-car1.hf.space:443
car	car2	https://oussama-bjm-car2.hf.space:443
notification	notification1	https://oussama-bjm-notification1.hf.space:443
notification	notification2	https://oussama-bjm-notification2.hf.space:443

7.7. RabbitMQ — Topologie des Exchanges et Queues

7.7.1. Exchanges configurés

Exchange	Type	Durabilité	Taux entrant/sortant
user_events	topic	Durable	0.01/s / 0.00/s
vehicle_events	topic	Durable	0.00/s / 0.00/s
notification_events	topic	Durable	0.00/s / 0.00/s

7.7.2. Connexions actives

Adresse client	Utilisateur	État	Protocol
3.212.25.85:43616	guest	running	AMQP 0-9-1
3.216.155.203:9860	guest	running	AMQP 0-9-1
3.228.31.30:48467	guest	running	AMQP 0-9-1
50.17.198.149:11178	guest	running	AMQP 0-9-1

Interprétation des métriques :

- Les **135 messages en file d'attente** (Ready) sont des notifications en attente d'être consommées
- **0 message non acquitté** indique que tous les messages sont correctement traités
- Les **12 consommateurs** répartis sur les 3 services assurent le traitement parallèle
- Les connexions utilisent le protocole **AMQP 0-9-1** avec identifiants guest/guest

7.8. Architecture Complète — Récapitulatif

Traefik 3.0.4	→	Gateway + Load Balancing
Consul v1.15.4	→	Service Registry + Health Checks
RabbitMQ	→	Messagerie asynchrone
3 services × 2 instances	→	6 instances au total

7.9. Procédure de déploiement

1. Démarrage de **RabbitMQ** et **Consul** sur leurs serveurs dédiés.
2. Déploiement des **2 instances** de l'**Auth Service** sur Hugging Face Spaces.
3. Déploiement des **2 instances** du **Catalog Service** + lancement du consumer sur chaque instance :
`python manage.py consume_user_events`
4. Déploiement des **2 instances** du **Notification Service** + lancement du worker sur chaque instance :
`python manage.py start_worker`
5. Configuration de **Traefik** comme gateway frontale avec discovery via Consul.
6. Enregistrement de toutes les instances dans Consul avec health checks.
7. Tests de la communication synchrone (Traefik → Consul → instances) et asynchrone (RabbitMQ entre services).

Haute Disponibilité et Tolérance aux Pannes

L'architecture à 6 instances garantit :

- Si une instance Auth tombe, l'autre prend le relais.
- Si une instance Catalog est surchargée, Traefik distribue les requêtes.
- Les messages RabbitMQ sont persistants et durables — aucune perte lors d'un redémarrage.
- Consul vérifie la santé de chaque instance toutes les 10 secondes.

8. Tests et Validation

8.1. Collection Postman

La collection Postman couvre l'ensemble des flux fonctionnels et des cas d'erreur, accompagnée d'un environnement de variables.

Scénario	Endpoints testés
Cycle d'authentification	Tests couvrant tous les endpoints d'authentification (signup, login, profile, change-password, logout, refresh)
CRUD véhicules	Tests CRUD complets sur l'API catalogue (création, lecture, mise à jour, suppression)
Gestion d'images	add_image, set_primary_image, remove-image
Système de follow	follow/unfollow user et post, consultation listes
Notifications	Lecture liste, marquage lu, tous lus, suppression
Contrôle d'accès	Modification d'un véhicule tiers → HTTP 403
Token invalide	Requête sans token ou expiré → HTTP 401
Ressource absente	Véhicule inexistant → HTTP 404

8.2. Vérification de la communication asynchrone

1. **Test user.created** — Créer un utilisateur, vérifier l'apparition du `UserModel` dans les bases du Catalog et du Notification Service.
2. **Test content_event** — Créer un véhicule ACTIVE, vérifier que les followers du créateur ont reçu une notification.
3. **Test vehicle.deleted** — Supprimer un véhicule, vérifier la suppression en cascade des `User-FollowPost` et `Notification` associés.
4. **Test user.deleted** — Supprimer un compte, vérifier la suppression en cascade dans tous les services.

9. Conclusion

9.1. Bilan technique

Le projet iCar démontre la mise en œuvre concrète d'une architecture microservices Django répondant à l'ensemble des exigences du cahier des charges WAMS :

- **Isolation des services** — chaque service possède sa propre base de données et peut évoluer indépendamment.
- **Authentification stateless** — la validation JWT locale élimine le couplage entre services pour chaque requête.
- **Consistance à terme** — la synchronisation via RabbitMQ garantit que chaque service dispose localement des données nécessaires.
- **Résilience** — si le Notification Service est indisponible, les fonctionnalités de base continuent de fonctionner normalement.
- **Intégrité des données** — transactions atomiques, signaux `pre_delete` et suppressions en cascade garantissent l'absence de données orphelines.

9.2. Grille d'évaluation

Conception	25%
Implémentation	30%
Déploiement multi-serveurs	20%
Asynchrone + Sécurité	15%
Présentation + Documentation	10%

9.3. Points livrés

- ✓ Service d'authentification JWT dédié avec token personnalisé, blacklist, rôles et publication RabbitMQ
- ✓ API REST catalogue complète avec 11 endpoints et validation métier stricte
- ✓ Authentification stateless sans accès base de données
- ✓ Communication asynchrone via deux types d'échange (topic et direct), messages persistants
- ✓ Nettoyage en cascade des données et des fichiers physiques
- ✓ Déploiement multi-serveurs sur Hugging Face Spaces avec Traefik et Consul
- ✓ Collection Postman complète avec environnement de variables